

Develop NLP Apps with Azure AI Language

Azure provides nlp through language services enabling developers to incorporate these advanced features into their data application

→ Case Study :Azure AI Language Service:

- My friend Tina got a job at a smart home device company
- The company's products are rapidly gaining popularity in the marketplace
- Customer service agents are overwhelmed with questions and support requests
- Tina has been assigned to lead a team to develop an AI solution

Project Goals

Goals	VS.	Capabilities
Extract key details to identify the core issue and categorize the inquiry appropriately		Azure AI Language capabilities to extract key phrases, recognize named entities and PII
Help to understand customer issues, by summarizing long emails		Text Summarization features of Azure AI Language
Address issues across a multi-language customer base		Cross-lingual capabilities of Azure AI Language

DEMO

- Extract key details to identify the core issue and categorize the inquiry appropriately
- Deploy an Azure AI Language resource
- Extract key phrases
- Extract custom named entities
- Implement entity linking
- Extract personal identifiable information

→ Azure AI Language Resource

- Key Phrase Extraction: Automatically identifies the main concepts or topics within text.
- Named Entity Recognition : Identifies and classifies real-world objects like people, locations, and organizations in unstructured data.
- Entity Linking
- PII Extraction : Detects and redacts Personally Identifiable Information to ensure data
- Summarization
- Language Detection

→ Create language by going to azure ai studio

[Home](#) > [Azure AI services](#) | [Language service](#) > [Select additional features](#) >

Create Language ...

[view full pricing details](#)

Custom text classification, Custom named entity recognition, Custom summarization, Custom sentiment analysis & Custom Text Analytics for health

You need to create your own storage when using these customization features to host and upload all your training and labeled data and have it saved in your own Azure subscription. You get to associate only one storage account with one language resource that cannot be changed moving forward.

[Learn more](#)

☐ New/Existing storage account *

☐ New storage account

☒ Existing storage account

Storage account in current selected subscription and resource region *

storaccallangsvc

[Home](#) > [Azure AI services](#) | [Language service](#) > [Select additional features](#) >

Create Language ...

Basics Network Identity Tags Review + create

Configure network security for your Azure AI services resource.

- ☒ All networks, including the internet, can access this resource.
- ☐ Selected networks, configure network security for your Azure AI services resource.
- ☐ Disabled, no networks can access this resource. You could configure private endpoint connections that will be the exclusive way to access this resource.

Virtual network

(New) vnet01 (AllLanguage_RG)

[Edit virtual network](#)

Subnets *

(New) subnet-1

[Edit subnet](#)

172.16.0.0 - 172.16.0.63 (64 addresses)

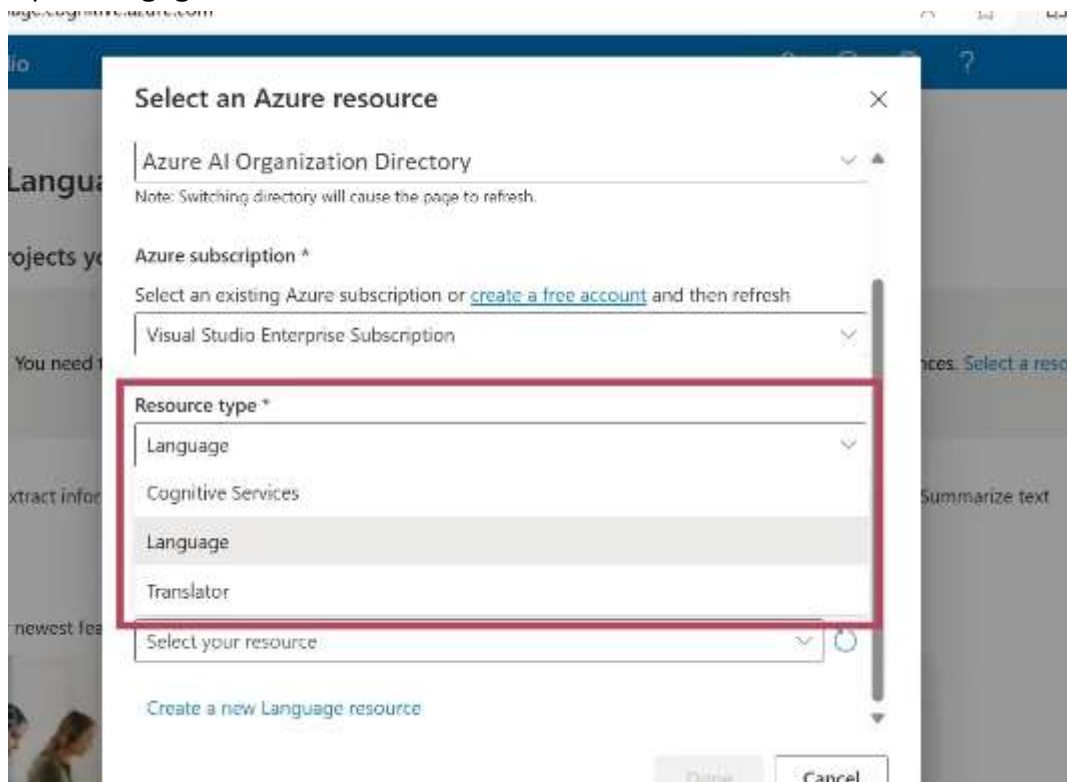
[Previous](#)

[Next](#)

[Review + create](#)



- ➔ After it is deployed you can use it
- ➔ Open Lmaguae studio on Azure AI



- ➔
- ➔ Use tool -> Extract key phrases
- ➔ Then you can upload and extract
- ➔ Then writing code on vscode
- ➔ First is Text Analysis
 - Import the required modules

```

2 # pip install python-dotenv
3
4 import sys
5 import os
6 import json
7 from dotenv import load_dotenv
8 from azure.ai.textanalytics import TextAnalyticsClient
9 from azure.core.credentials import AzureKeyCredential
10

```

- Creating a function endpoint for retrieve key and service endpoint from .env and for linking azure ai authentication
- The code to extract key phrase from the set of document

```

def extract_key_phrases(client, documents):
    try:
        response = client.extract_key_phrases(documents=documents)
        result = [doc for doc in response if not doc.is_error]
        return result
    except Exception as err:
        print(f"Encountered exception. {err}")
        return []

```

Function

Purpose: The extract_key_phrases function is designed to take a client

object and a list of documents, then process them to find significant phrases [1].

- **Key Phrase Extraction:** The highlighted line (`response = client.extract_key_phrases(...)`) actually calls the service to perform the analysis on the provided documents [1].
 - **Error Handling:** The `try...except` block is included to catch any exceptions that might occur during the API call, printing the error and returning an empty list if a failure occurs [1].
 - **Result Filtering:** The `result = [...]` line filters the API response to include only successfully processed documents, excluding those that resulted in an error [1].
- Python script defines a `main()` function that orchestrates an email processing workflow, likely using an Azure Cognitive Services SDK for key phrase extraction.

```
# Main function to tie it all together
def main():
    emails = getMessages()
    # Transform emails into the required document format with id, text, and optional language
    documents = [{"id": str(i), "text": email["body"], "subject": email["subject"], "language": "en"} for i, email in enumerate(emails)]

    client = authenticate_client()
    results = extract_key_phrases(client, documents)

    serialized_result = []
    for result in results:
        email_id = int(result.id) # Convert id back to integer to access the original email
        serialized_result.append({
            "Email id": result.id,
            "Subject": emails[email_id]["subject"],
            "key_phrase": result.key_phrases
        })

    with open(output_file, 'w') as f:
        json.dump(serialized_result, f, indent=4)

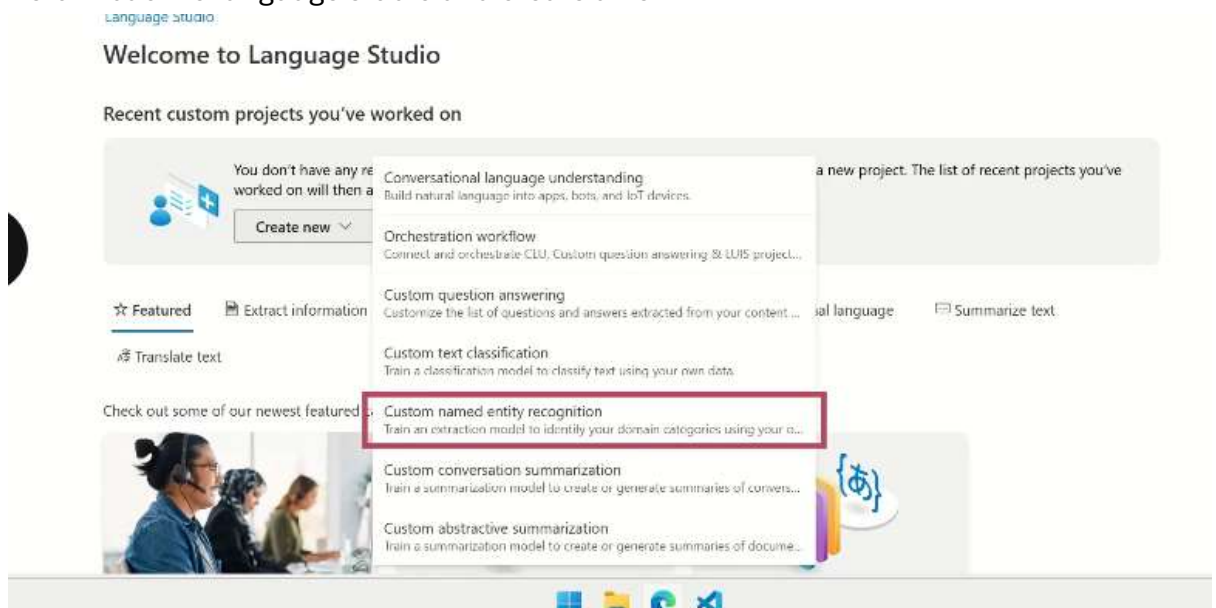
output_file = os.path.join('Results', 'key_phrases_result.json')
main()
```

- **Data Retrieval:** It calls `getMessages()` to fetch a list of email objects.
- **Data Transformation:** It converts those emails into a specific documents format (a list of dictionaries containing id, text, subject, and language) required by the API.
- **Processing:**
 - Authenticates a client.
 - Sends the documents to `extract_key_phrases()` to analyze the text.
- **Serialization:**
 - Iterates through the results.

- Maps the analysis back to the original email data using the id.
- Constructs a new list of dictionaries containing the Email ID, Subject, and the extracted Key Phrases.
- **Output:** Saves the final structured data into a JSON file located at Results/key_phrases_result.json

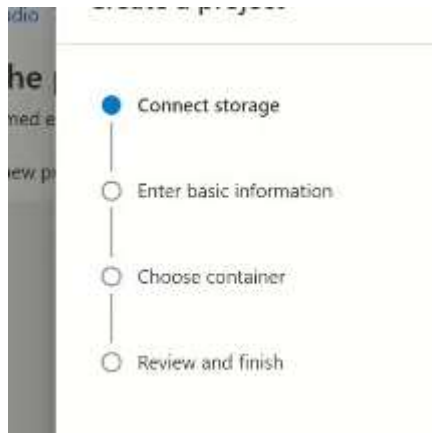
➔ Language service provide entity recognition , we can navigate to named entity recognition

➔ Return back to language studio and create a new



- **Conversation Understanding:** Features conversational language understanding (CLU) to help build interactive bots and natural language applications.
- **Question Answering:** Includes tools for customizing how bots extract answers from content.
- **Text Analysis:** Allows for custom named entity recognition and classification to train models to identify domain-specific categories. [1, 2, 3]

➔ It is a four stage process :



- ➔ Labelling you input under some label categories and dividing them for extracting testing training data differently
- ➔ Training the job

Language Studio > Custom Named Entity Recognition > ProductCatalogProject - Training jobs

Start a training job

ProductCatalogModel

☐ Overwrite an existing model

Data splitting

We use separate datasets to train your model and test its accuracy. [Learn more about data splitting.](#)

☐ Automatically split the testing set from training data

We'll select stratified samples from all training data according to the percentages that you provide here. Any data already assigned to the testing set will be ignored completely.

80 % for training
20 % for testing

☒ Use a manual split of training and testing data

We'll use the training and testing sets that you've assigned in [Data Labeling](#) to create your custom model and measure its performance.

Your current split of data
66.67% training 33.33% testing
[View distribution of data](#)

Train Cancel

- ➔ Adding the product created

```
28 # Step 3: extract custom named entities
29 def extract_custom_named_entities(client, documents):
30     try:
31         operation = client.begin_recognize_custom_entities(
32             documents,
33             project_name="ProductCatalogProject",
34             deployment_name="ProductCatalogDeployment"
35         )
36     except Exception as e:
```

Demo: Help to understand customers issues by summarizing long emails

- ➔ Implementing and extracting summary from email

→ This can be done in two ways:

Azure AI Language Summarization

Extractive	VS.	Abstractive
Summarizes text by extracting sentences that best represent the key information from the original text		Produces a summary using concise and coherent sentences that are not direct extracts from the original content
Extractive summaries are direct excerpts		Abstractive summaries are newly generated sentences

→ Open language studio again and navigate to summarization information tryout

→ And you can summarize the text

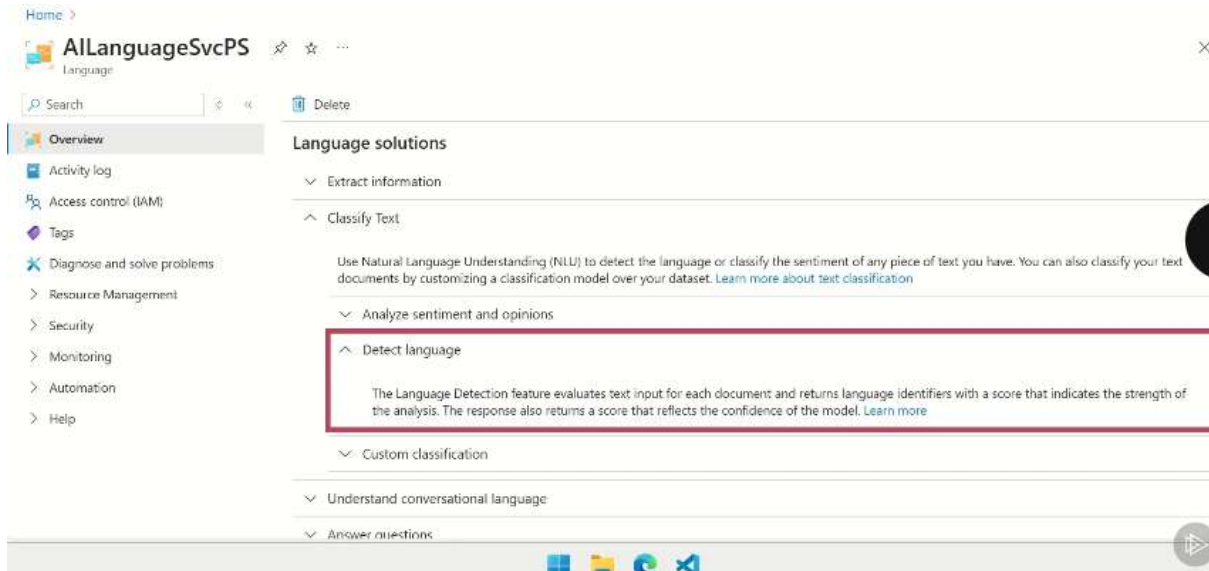
- Then include this features within project

```
2. Summarization > summarization.py > main
27
28 def summarization(client, documents):
29     try:
30         poller = client.begin_abstract_summary(documents)
31         result = poller.result()
32         docs = [doc for doc in result if not doc.is_error]
33
34         results = []
35         for i, result in enumerate(docs, start=1): # Start indexing from 1
36             if result.kind == "AbstractiveSummarization":
37                 summary = " ".join([summary.text for summary in result.summaries])
38                 results.append({
39                     "Email id": i,
40                     "Subject": next(d['subject'] for d in documents if d['id'] == result.id),
41                     "Summary": summary
42                 })
43             elif result.is_error:
44                 print(f"Error: {result.error.code} - {result.error.message}")
```

- **Asynchronous Processing:** The code uses `client.begin_abstract_summary(documents)` to initiate an asynchronous operation, allowing the application to continue running while the service processes the documents. [1]
- **Result Retrieval:** `poller.result()` blocks execution until the summarization task is complete and retrieves the results. [1]
- **Data Structuring:** The code iterates through the successfully processed documents, extracts the generated summaries, and maps them back to the original document subjects using a dictionary format. [1]

- **Error Handling:** A basic error-checking mechanism (if result.is_error) is included to catch and print error codes and messages if the summarization fails for a specific document. [1]

Implementing text language detection capabilities:



Using using azure Ai language studio and there using can define the language that it has to be detecting in and passing the language and deciding the output



This detection of language can be used in email summarization and is used for detecting the language used in email and extracting the language content.